CS3400 – Trees and Hierarchical Data Structures

Lecture Notes

---

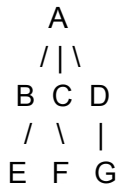
1. Introduction To Trees

1.1 What Is a Tree?

A tree is a non-linear data structure used to represent hierarchical relationships between elements.

Unlike linear data structures such as arrays, linked lists, stacks, and queues, a tree does not arrange elements in a single sequence. Instead, elements are organized into levels, with relationships between parent and child nodes.

For example:



In this tree:

- A is the root.
- B, C, and D are children of A.
- B is the parent of E and F.
- D is the parent of G.
- E, F, C, and G are leaf nodes.

Trees are useful for representing naturally hierarchical information.

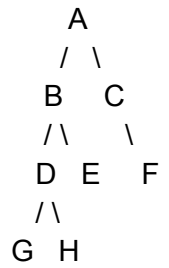
Examples include:

- File systems
- Organisation structures
- Family trees
- HTML/XML documents
- Database indexes
- Decision trees
- Expression trees
- Searching structures

The important characteristic of a tree is that it represents a hierarchical relationship between elements.

1.2 Tree Terminology

Consider the following tree:



Understanding tree terminology is essential because the terminology is used throughout the rest of this topic.

Node

A node is an individual element in a tree.

In the example above:

A, B, C, D, E, F, G, and H are all nodes.

A node usually contains two types of information:

1. Data
2. References or pointers to other nodes

For example, a binary tree node in C++ can be represented as:

```
struct Node {
    int data;
    Node* left;
    Node* right;
};
```

Edge

An edge is a connection between two nodes.

For example:

A -> B

represents an edge connecting A and B.

A tree containing n nodes always contains:

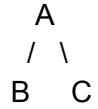
n - 1 edges

Therefore, a tree with 10 nodes has:

10 - 1 = 9 edges

Root

The root is the topmost node in a rooted tree.



A is the root.

A rooted tree has exactly one root.

Parent

A parent is a node that is directly connected to one or more nodes below it.



A is the parent of B and C.

Child

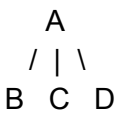
A child is a node directly below another node.

In the example above:

- B is a child of A.
- C is a child of A.

Siblings

Siblings are nodes that have the same parent.

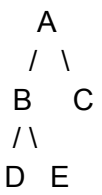


B, C, and D are siblings because they all have A as their parent.

Leaf Node

A leaf node is a node that has no children.

For example:



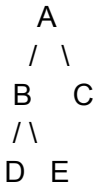
C, D, and E are leaf nodes.

A leaf is also called a terminal node.

Internal Node

An internal node is a node that has at least one child.

In:



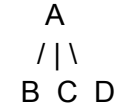
A and B are internal nodes.

C, D, and E are leaves.

1.3 Degree of a Node

The degree of a node is the number of children that the node has.

For example:



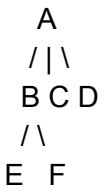
A has degree 3.

B, C, and D each have degree 0.

1.4 Degree of a Tree

The degree of a tree is the maximum degree of any node in the tree.

For example:



The maximum number of children belonging to any node is 3.

Therefore:

Degree of tree = 3

1.5 Path

A path is a sequence of nodes connected by edges.

For example:

A -> B -> D -> G

is a path from A to G.

1.6 Path Length

The length of a path is the number of edges in the path.

For:

A -> B -> D -> G

there are three edges:

A -> B
B -> D
D -> G

Therefore:

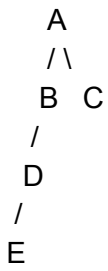
Path length = 3

Notice that the path contains four nodes but has a length of three.

1.7 Depth of a Node

The depth of a node is the number of edges from the root to that node.

Consider:



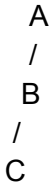
Using the conventional definition that the root has depth 0:

Depth(A) = 0
Depth(B) = 1
Depth(C) = 1
Depth(D) = 2
Depth(E) = 3

1.8 Height of a Node

The height of a node is the number of edges on the longest path from that node to a leaf.

Consider:



C is a leaf.

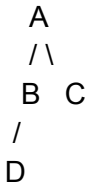
Therefore:

Height(C) = 0
Height(B) = 1
Height(A) = 2

1.9 Height of a Tree

The height of a tree is the height of its root.

For example:



The longest path from the root to a leaf is:

A -> B -> D

Therefore:

Height of tree = 2

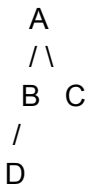
1.10 Depth Versus Height

Students frequently confuse depth and height.

Depth measures the distance FROM the root TO a node.

Height measures the longest distance FROM a node DOWN TO a leaf.

For example:



Depth(D) = 2

Height(B) = 1

Height(A) = 2

A useful way to remember the difference is:

Depth = distance downward from the root.

Height = distance downward to the deepest leaf.

1.11 Ancestors and Descendants

Consider:



A is an ancestor of B, C, and D.

B is an ancestor of C and D.

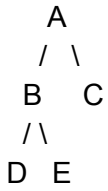
C is an ancestor of D.

D is a descendant of A, B, and C.

1.12 Subtree

A subtree is a tree formed by a node and all of its descendants.

Consider:



The subtree rooted at B is:



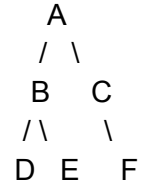
A tree can therefore be viewed as being composed of smaller trees called subtrees.

This idea is particularly important for recursive tree algorithms.

1.13 Rooted Trees

A rooted tree is a tree in which one node has been designated as the root.

For example:



A is the root.

Once a root is established, concepts such as:

- Parent
- Child
- Ancestor
- Descendant
- Depth
- Height

can be defined.

1.14 Properties of Trees

Several important properties apply to trees.

Property 1: A tree with n nodes has n - 1 edges.

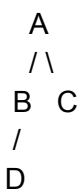
E = n - 1

For example, a tree with 20 nodes has:

19 edges

Property 2: There is exactly one path between any two nodes.

Consider:



There is exactly one path from D to C:

D -> B -> A -> C

Property 3: A tree contains no cycles.

A cycle would allow us to start at a node, follow a sequence of edges, and eventually return to the same node.

Trees do not contain cycles.

Property 4: Removing any edge disconnects the tree.

For example:

A -> B

If the edge connecting A and B is removed, the two nodes become disconnected.

1.15 Tree Representations

Trees can be represented in several ways.

Linked Representation

Each node stores data and references to other nodes.

For a binary tree:

```
struct Node {
   int data;
   Node* left;
   Node* right;
};
```

This representation is flexible because nodes can be dynamically created as required.

Parent Array Representation

A tree can also be represented using an array in which each element stores the parent of a node.

For example:

Node	Parent
A	-1
B	A
C	A
D	B
E	B

A parent value of -1 can indicate that the node is the root.

This representation is useful when parent relationships are important.

First-Child / Next-Sibling Representation

A general tree can also be represented using two pointers:

```
struct Node {
    int data;
    Node* firstChild;
    Node* nextSibling;
};
```

The first pointer identifies the node's first child.

The second pointer identifies the next sibling.

This allows a general tree with an arbitrary number of children to be represented using a fixed number of pointers.

1.16 Abstract Tree Structures

An Abstract Data Type describes what operations are available without specifying how those operations are implemented.

Possible Tree ADT operations include:

create()
isEmpty()
root()
parent(node)
children(node)
isLeaf(node)
height(node)
depth(node)
search(key)
insert(key)
remove(key)
traverse()

The important distinction is:

ADT = What the structure does.

Data structure = How the structure is implemented.

For example, a tree ADT might be implemented using:

- Linked nodes
- Arrays
- Parent arrays
- Binary trees
- Binary search trees
- AVL trees
- B-trees

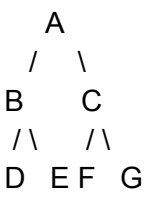
---

2. Tree and Traversals

2.1 What Is Tree Traversal?

Tree traversal is the process of visiting every node in a tree according to a specific ordering.

Consider:



Different traversal techniques produce different sequences.

The major traversal methods are:

1. Preorder
2. Inorder
3. Postorder
4. Breadth-first or level-order

Preorder, inorder, and postorder are depth-first traversals.

2.2 Depth-First Traversal

Depth-first traversal explores one branch as deeply as possible before returning to explore another branch.

Conceptually:

1. Start at a node.
2. Explore downward.
3. Continue until there is nowhere further to go.
4. Backtrack.
5. Explore the next branch.

There are three common depth-first traversals:

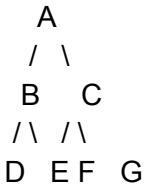
- Preorder
- Inorder
- Postorder

2.3 Preorder Traversal

Preorder follows:

Root -> Left -> Right

For:



The preorder traversal is:

A B D E C F G

The process is:

1. Visit A.
2. Traverse the left subtree.
3. Traverse the right subtree.

Pseudocode

```
PREORDER(node)

   if node is NULL
      return

   visit(node)

   PREORDER(node.left)

   PREORDER(node.right)
```

C++ Implementation

```
void preorder(Node* node) {
   if (node == nullptr)
      return;

   cout << node->data << " ";

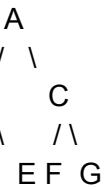
   preorder(node->left);
   preorder(node->right);
}
```

2.4 Inorder Traversal

Inorder follows:

Left -> Root -> Right

For:



The inorder traversal is:

D B E A F C G

Pseudocode

```
INORDER(node)

   if node is NULL
      return

   INORDER(node.left)

   visit(node)

   INORDER(node.right)
```

C++ Implementation

```
void inorder(Node* node) {
   if (node == nullptr)
      return;

   inorder(node->left);

   cout << node->data << " ";

   inorder(node->right);
}
```

An important property of a Binary Search Tree is:

> Inorder traversal visits the keys in sorted order.

For example:

```
          50
        /    \
      30      70
     / \     / \
    20 40   60 80
```

Inorder traversal:

20 30 40 50 60 70 80

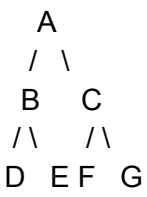
This is one of the most important applications of inorder traversal.

2.5 Postorder Traversal

Postorder follows:

Left -> Right -> Root

For:



The postorder traversal is:

D E B F G C A

Pseudocode

```
POSTORDER(node)

   if node is NULL
      return

   POSTORDER(node.left)

   POSTORDER(node.right)

   visit(node)
```

C++ Implementation

```
void postorder(Node* node) {
   if (node == nullptr)
      return;

   postorder(node->left);
   postorder(node->right);

   cout << node->data << " ";
}
```

Postorder is useful when children need to be processed before their parent.

For example, when deleting an entire tree, we generally want to delete the children before deleting the parent.

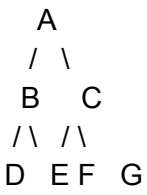
2.6 Breadth-First Traversal

Breadth-first traversal visits nodes level by level.

It is also called:

Level-order traversal.

For:



The level-order traversal is:

A B C D E F G

The algorithm uses a queue.

Pseudocode

```
BFS(root)

    create an empty queue

    enqueue root

    while queue is not empty

        node = dequeue()

        visit(node)

        if node has a left child
            enqueue left child

        if node has a right child
```

enqueue right child

C++ Implementation

```
#include <queue>

void levelOrder(Node* root) {
    if (root == nullptr)
        return;

    queue<Node*> q;

    q.push(root);

    while (!q.empty()) {

        Node* current = q.front();
        q.pop();

        cout << current->data << " ";

        if (current->left != nullptr)
            q.push(current->left);

        if (current->right != nullptr)
            q.push(current->right);
    }
}
```

2.7 Traversal Complexity

Suppose the tree contains n nodes.

Every traversal must visit every node.

Therefore:

Time complexity = O(n)

For recursive depth-first traversal, the additional space is related to the height of the tree.

Space complexity:

O(h)

where h is the height of the tree.

For breadth-first traversal, the space required depends on the maximum number of nodes stored in the queue at one time.

2.8 Comparing Tree Traversals

Traversal	Ordering
Preorder	Root, Left, Right
Inorder	Left, Root, Right
Postorder	Left, Right, Root
Level-order	Level by level

A useful memory aid:

Preorder:

ROOT comes first.

Inorder:

ROOT comes in the middle.

Postorder:

ROOT comes last.

---

3. Binary Trees And Binary Search Trees

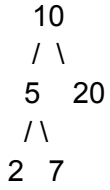
3.1 Binary Trees

A binary tree is a tree in which each node has at most two children.

The two children are called:

- Left child
- Right child

Example:



Every node has zero, one, or two children.

3.2 Binary Tree Node Representation

A binary tree node can be represented in C++ as:

```
struct Node {
    int data;
    Node* left;
    Node* right;
};
```

A function can be used to create a new node:

```
Node* createNode(int value) {

    Node* node = new Node;

    node->data = value;
    node->left = nullptr;
    node->right = nullptr;

    return node;
}
```

Because the node is dynamically allocated, memory management must be considered carefully.

This connects directly to the C++ concepts of:

- Dynamic memory
- Pointers
- Object lifetime
- RAII
- Memory leaks

3.3 Binary Tree Properties

Maximum Number of Nodes at Depth d

The maximum number of nodes at depth d is:

2^d

Therefore:

Depth 0 = 1 node

Depth 1 = 2 nodes

Depth 2 = 4 nodes

Depth 3 = 8 nodes

Maximum Number of Nodes at Height h

The maximum number of nodes in a binary tree of height h is:

2^(h+1) - 1

For example, if:

h = 3

then:

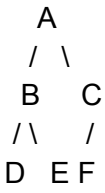
Maximum nodes = 2^4 - 1
= 16 - 1
= 15

3.4 Complete Binary Tree

A complete binary tree is a binary tree in which:

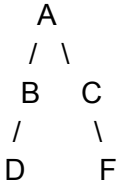
- Every level except possibly the last is completely filled.
- The final level is filled from left to right.

Example:



This is a complete binary tree.

However:



is not complete because the final level is not filled from left to right.

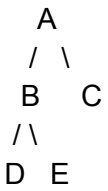
Complete binary trees are especially important when studying heaps.

3.5 Full Binary Tree

A full binary tree is a binary tree in which every node has either:

- Zero children, or
- Two children.

Example:



A and B have two children.

C, D, and E have zero children.

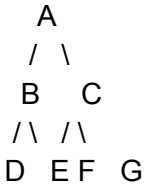
Therefore, the tree is full.

3.6 Perfect Binary Tree

A perfect binary tree has:

1. Every internal node with exactly two children.
2. All leaf nodes at the same depth.

Example:



For a perfect binary tree of height h:

n = 2^(h+1) - 1

3.7 Array Representation of Binary Trees

A binary tree can be represented using an array.

This works particularly well for complete binary trees.

Using zero-based indexing:

Left child of node at index i:

2i + 1

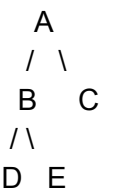
Right child:

2i + 2

Parent:

(i - 1) / 2

For example:



can be stored as:

Index: 0 1 2 3 4
Value: A B C D E

This representation is especially useful for heaps.

3.8 Binary Search Tree

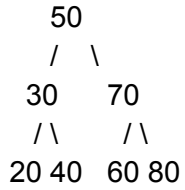
A Binary Search Tree, or BST, is a binary tree that maintains an ordering relationship between the keys.

For every node:

All keys in the left subtree are smaller than the node's key.

All keys in the right subtree are larger than the node's key.

For example:



For node 50:

Left subtree:

20, 30, 40

All are less than 50.

Right subtree:

60, 70, 80

All are greater than 50.

3.9 BST ADT

A Binary Search Tree ADT may provide operations such as:

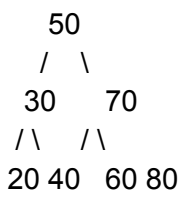
insert(key)
search(key)
remove(key)
findMin()
findMax()
isEmpty()
traverse()

The BST ADT specifies what operations are available.

The actual implementation can use dynamically allocated nodes connected through pointers.

3.10 Searching a BST

Consider:



Suppose we want to search for 60.

First compare 60 with 50.

60 > 50

Therefore, search the right subtree.

Compare 60 with 70.

60 < 70

Therefore, search the left subtree.

We find 60.

The search path is:

50 -> 70 -> 60

We did not have to examine every node.

3.11 BST Search Algorithm

```
SEARCH(node, key)

    if node is NULL
        return false

    if key == node.key
        return true

    if key < node.key
        return SEARCH(node.left, key)

    return SEARCH(node.right, key)
```

C++ Implementation

```
bool search(Node* node, int key) {

    if (node == nullptr)
        return false;

    if (key == node->data)
        return true;

    if (key < node->data)
        return search(node->left, key);

    return search(node->right, key);
}
```

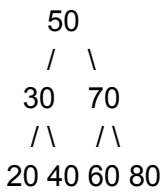
3.12 BST Insertion

Insertion follows the same comparison process as searching.

Consider inserting:

65

into:



Compare 65 with 50:

65 > 50

Move right.

Compare 65 with 70:

65 < 70

Move left.

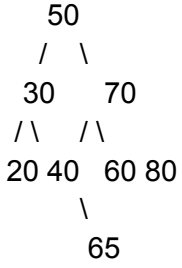
Compare 65 with 60:

65 > 60

Move right.

The right position of 60 is empty, so 65 is inserted there.

Result:



3.13 BST Insertion – C++

```
Node* insert(Node* node, int key) {

    if (node == nullptr)
        return createNode(key);

    if (key < node->data)
        node->left = insert(node->left, key);

    else if (key > node->data)
        node->right = insert(node->right, key);

    return node;
}
```

The recursive structure works naturally because each child is itself the root of a subtree.

3.14 BST Deletion

Deletion is the most complicated basic BST operation.

There are three cases.

Case 1: Node Has No Children

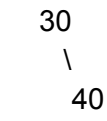
Suppose:

```
        30
        /  \
      20   40
```

We delete 20.

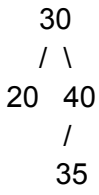
Because 20 is a leaf, we simply remove it.

Result:



Case 2: Node Has One Child

Suppose:

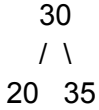


Delete 40.

40 has one child, 35.

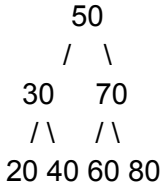
The child takes the position of 40.

Result:



Case 3: Node Has Two Children

Consider:



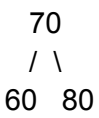
Suppose we delete 50.

We cannot simply remove 50 because it has two children.

One common approach is to replace it with its inorder successor.

The inorder successor is the smallest key in the right subtree.

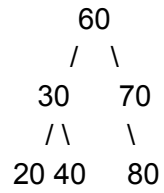
The right subtree is:



The smallest value is 60.

Therefore, replace 50 with 60.

Result:



The other option is to use the inorder predecessor, which is the largest key in the left subtree.

3.15 Finding the Minimum in a BST

The minimum value is always found by following left children until there is no further left child.

```
Node* findMin(Node* node) {

   while (node->left != nullptr)
      node = node->left;

   return node;
}
```

Similarly, the maximum value is found by following right children.

```
Node* findMax(Node* node) {

    while (node->right != nullptr)
        node = node->right;

    return node;
}
```

3.16 BST Complexity

BST operations depend on the height of the tree.

Let:

h = height of the tree

Then:

Search = O(h)

Insertion = O(h)

Deletion = O(h)

If the tree is reasonably balanced:

h = O(log n)

Therefore:

Search = O(log n)

Insertion = O(log n)

Deletion = O(log n)

However, a BST can become badly skewed.

3.17 Worst-Case BST

Suppose we insert the values:

10, 20, 30, 40, 50

The tree becomes:

10

 20

 30

 40

 50

The tree has effectively become a linked list.

The height is approximately:

h = n - 1

Therefore:

Search = O(n)

Insertion = O(n)

Deletion = O(n)

This is a major weakness of an ordinary BST.

This problem motivates the use of balanced search trees.

---

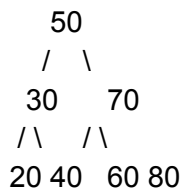
4. Balanced Search Trees

4.1 Motivation for Balancing

A BST is efficient when its height is small.

Compare:

Balanced tree:



with:

Skewed tree:

50

70

80

90

Both are valid BSTs.

However, the first tree has logarithmic height while the second can have linear height.

Therefore, we want search trees that automatically maintain a small height.

One such structure is the AVL tree.

4.2 AVL Trees

An AVL tree is a self-balancing Binary Search Tree.

AVL stands for:

Adelson-Velsky and Landis.

The main rule is:

For every node, the heights of the left and right subtrees may differ by at most 1.

Therefore:

Balance Factor	Meaning
-1	Right subtree slightly higher
0	Equal heights
+1	Left subtree slightly higher

A node is considered balanced when its balance factor is:

-1, 0, or +1.

4.3 Balance Factor

The balance factor is calculated as:

Balance Factor = Height(left subtree) - Height(right subtree)

For example:

```
    30
   /  \
  20   40
```

The left and right subtrees have equal height.

Therefore:

BF(30) = 0

Consider:

```
      30
     /
    20
   /
  10
```

The left subtree is significantly taller than the right subtree.

Therefore:

BF(30) = +2

The node is unbalanced.

4.4 AVL Balance Condition

For every node in an AVL tree:

|Balance Factor| <= 1

Therefore, valid balance factors are:

-1
 0
 +1

If:

BF = +2

the tree is too heavy on the left.

If:

BF = -2

the tree is too heavy on the right.

A rotation is required.

4.5 Rotations

AVL trees use rotations to restore balance.

There are four cases:

1. LL
2. RR
3. LR
4. RL

The appropriate rotation depends on the direction in which the tree became unbalanced.

4.6 LL Imbalance – Right Rotation

Suppose we insert:

30, 20, 10

The tree becomes:

```
   30
   /
  20
 /
10
```

The tree is left-heavy.

This is an LL imbalance.

Perform a right rotation:

```
   20
   /  \
  10   30
```

The tree is balanced again.

4.7 RR Imbalance – Left Rotation

Suppose we insert:

10, 20, 30

The tree becomes:

10

 20

 30

This is an RR imbalance.

Perform a left rotation:

```
  20
 /  \
10   30
```

4.8 LR Imbalance

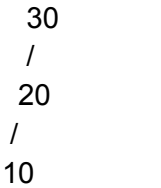
Consider:

```
   30
  /
 10
   \
    20
```

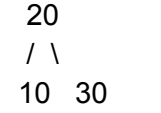
This is a Left-Right imbalance.

Two rotations are required.

First, perform a left rotation on 10:

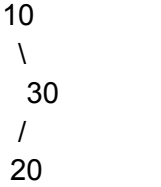


Then perform a right rotation on 30:



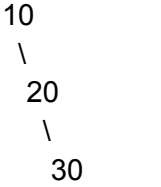
4.9 RL Imbalance

Consider:

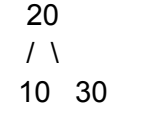


This is a Right-Left imbalance.

First perform a right rotation on 30:



Then perform a left rotation on 10:



4.10 AVL Rotation Summary

Imbalance	Pattern	Required Rotation
LL	Left-Left	Right rotation
RR	Right-Right	Left rotation
LR	Left-Right	Left rotation followed by Right rotation
RL	Right-Left	Right rotation followed by Left rotation

Students should be able to identify the pattern before deciding which rotation to perform.

4.11 Why Do Rotations Work?

A rotation changes the shape of the tree without violating the BST ordering property.

For example:

```
   30
   /
  20
 /
10
```

becomes:

```
   20
   /  \
 10   30
```

The ordering is still:

10 < 20 < 30

but the tree is now shorter and better balanced.

This is the key purpose of AVL rotations.

4.12 AVL Insertion

AVL insertion begins with ordinary BST insertion.

The process is:

1. Insert the new key using BST rules.
2. Move back toward the root.
3. Update the heights.
4. Calculate balance factors.
5. Find any unbalanced nodes.
6. Perform the appropriate rotations.

For example, insert:

30, 20, 10

Initially:

```
   30
   /
  20
 /
10
```

Node 30 becomes unbalanced.

This is an LL case.

Perform a right rotation:

```
   20
   /  \
  10   30
```

4.13 Conceptual AVL Insertion Algorithm

```
AVL_INSERT(node, key)

    if node is NULL
        create new node
        return node

    if key < node.key
        node.left = AVL_INSERT(node.left, key)

    else if key > node.key
        node.right = AVL_INSERT(node.right, key)

    update node height

    calculate balance factor

    if balance > 1 and key < node.left.key
        perform right rotation

    if balance < -1 and key > node.right.key
        perform left rotation

    if balance > 1 and key > node.left.key
        perform left rotation on left child
        perform right rotation

    if balance < -1 and key < node.right.key
        perform right rotation on right child
        perform left rotation

    return node
```

4.14 AVL Node Representation in C++

An AVL node needs additional information to maintain its height.

For example:

```
struct AVLNode {
    int data;
    int height;

    AVLNode* left;
    AVLNode* right;
};
```

The height field allows the implementation to calculate balance factors efficiently.

4.15 AVL Deletion

AVL deletion begins with normal BST deletion.

However, after deletion, the tree may become unbalanced.

The process is:

1. Locate the node.
2. Perform BST deletion.
3. Update heights.
4. Calculate balance factors.
5. Perform rotations if necessary.
6. Continue checking ancestors toward the root.

Deletion can be more complicated than insertion because removing one node may affect the balance of several ancestors.

4.16 AVL Complexity

An AVL tree maintains logarithmic height.

Therefore:

Height = O(log n)

Consequently:

Operation	AVL Complexity
Search	O(log n)
Insertion	O(log n)
Deletion	O(log n)
Find minimum	O(log n)
Find maximum	O(log n)
Traversal	O(n)

The major advantage is that these are guaranteed worst-case bounds.

---

5. M-WAY SEARCH TREES AND B-TREES

5.1 Motivation for Multiway Trees

A binary tree allows each node to have at most two children.

However, this is not always the most efficient structure.

Suppose a tree is stored on secondary storage.

Accessing secondary storage is significantly more expensive than accessing data already in main memory.

Therefore, it is desirable to reduce the number of times we need to access storage.

A multiway tree addresses this problem by allowing each node to contain several keys and several children.

5.2 M-Way Search Trees

An M-way search tree is a search tree in which each node can have at most m children.

If a node has m children, it can contain at most:

m - 1 keys

For example, a 4-way search tree can contain at most:

4 children

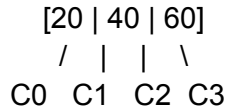
and:

3 keys

A node could look like:

[20 | 40 | 60]

with four children:



The keys divide the possible search values into ranges.

C0 contains values less than 20.

C1 contains values between 20 and 40.

C2 contains values between 40 and 60.

C3 contains values greater than 60.

5.3 Why M-Way Search Trees?

Consider a binary search tree.

Each node contains one key and can have at most two children.

A multiway tree can store several keys in one node.

For example:

Binary tree node:

[50]

Multiway node:

[20 | 40 | 60 | 80]

A single node can therefore represent several search divisions.

This can significantly reduce the height of the tree.

5.4 B-Trees

A B-tree is a balanced multiway search tree designed for efficient storage and retrieval of large amounts of data.

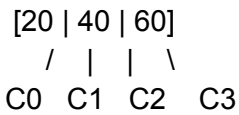
B-trees are particularly important in:

- Database systems
- File systems
- Storage systems
- Database indexing
- Large external data sets

The main advantage of a B-tree is that it keeps the tree shallow while allowing each node to contain multiple keys.

5.5 B-Tree Structure

Consider:



The keys divide the search space.

C0:

keys < 20

C1:

20 < keys < 40

C2:

40 < keys < 60

C3:

keys > 60

The keys inside a node are stored in sorted order.

5.6 B-Tree Properties

The exact terminology for B-tree order varies between textbooks, so the convention being used should always be stated.

Using the common definition where an order-m B-tree has at most m children:

1. Each node has at most m children.
2. Each node has at most m - 1 keys.
3. Except for the root, internal nodes have a minimum number of children determined by the B-tree order.
4. Keys within a node are sorted.
5. The children represent different key ranges.
6. All leaves occur at the same depth.
7. The tree remains balanced after insertion and deletion.
8. The root has special rules regarding its minimum occupancy.

The most important property is:

All leaves are at the same level.

Therefore, a B-tree remains balanced.

5.7 Searching a B-Tree

Consider:

```
 [20 | 40 | 60]
 /   |   |   \
C0   C1   C2   C3
```

Suppose we search for 50.

Compare 50 with 20.

50 > 20.

Compare 50 with 40.

50 > 40.

Compare 50 with 60.

50 < 60.

Therefore, follow C2.

The process continues in the selected child.

The search eliminates large portions of the tree at each level.

5.8 B-Tree Insertion

B-tree insertion is more complicated than ordinary BST insertion because nodes have a maximum capacity.

Suppose a node is full:

[10 | 20 | 30 | 40]

and another key must be inserted.

The node cannot simply grow indefinitely.

Instead, the node is split.

Conceptually:

```
      [30]
      /   \
[10 | 20] [40 | 50]
```

The middle key is promoted to the parent.

This process maintains the B-tree properties.

5.9 Node Splitting

Consider:

```
        [50]
        /   \
  [10|20|30] [60|70]
```

Suppose another key is inserted into the left node and causes it to become overfull.

The node may be split and a key promoted to the parent.

Conceptually:

```
        [20 | 50]
         /   |   \
   [10]  [30]  [60|70]
```

The exact key promoted depends on the B-tree order and the insertion procedure being used.

5.10 Root Splitting

The root can also become full.

When the root is split, a new root is created.

For example:

Before:

[10 | 20 | 30 | 40]

After:

```
        [30]
        /   \
    [10|20] [40]
```

The height of the tree increases by one.

The important point is that all leaves remain at the same depth.

5.11 B-Tree Deletion

B-tree deletion is more complicated because removing a key can cause a node to contain fewer keys than allowed.

Several operations may be required.

Borrowing or Redistribution

If a neighbouring sibling has additional keys, a key can be redistributed.

The parent separator may also need to be adjusted.

Merging

If neighbouring nodes do not have enough keys to lend, nodes may be merged.

For example, two nodes can be combined with a separator key from their parent.

Conceptually:

[10] [30] [50]

may become:

[10 | 30 | 50]

depending on the exact deletion situation.

Propagating the Problem

A merge can cause the parent to become deficient.

The same process may then have to continue upward toward the root.

This is why B-tree deletion is considerably more complicated than BST deletion.

5.12 Why Are B-Trees Used in Databases?

Consider a database containing millions of records.

Suppose we want to locate:

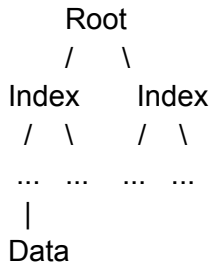
Student Number = 202301234

Searching every record sequentially would be extremely inefficient.

A database can maintain an index using a B-tree or a closely related structure.

Instead of examining millions of records, the index progressively narrows the search.

Conceptually:

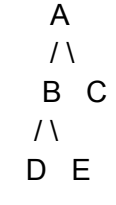


At every level, the search is narrowed to a smaller range.

5.13 B-Trees and Secondary Storage

The most important practical reason for B-trees is their suitability for secondary storage.

Consider a binary tree:



If every node requires a separate storage access, a large tree could require many accesses.

A B-tree instead stores many keys in one node:

[20 | 40 | 60 | 80 | 100 | 120]

One storage access can therefore retrieve many keys.

This reduces the number of storage accesses required to locate a particular record.

Therefore:

> B-trees are designed to minimize the number of expensive secondary-storage accesses.

This is why B-trees are so important in database indexing.

5.14 B-Tree Complexity

For a B-tree containing n keys:

Height = O(log n)

Therefore, basic search, insertion, and deletion operations have logarithmic asymptotic complexity.

Operation	Complexity
Search	O(log n)
Insertion	O(log n)
Deletion	O(log n)

However, the practical advantage of a B-tree is not simply the fact that it is O(log n).

Its major advantage is that the branching factor is large, making the tree very shallow and reducing the number of external storage accesses.

5.15 BST vs AVL vs B-Tree

Feature	BST	AVL Tree	B-Tree
Maximum children	2	2	Many
Keys per node	1	1	Multiple
Automatically balanced	No	Yes	Yes
Worst-case search	O(n)	O(log n)	O(log n)
Rotations	No	Yes	No
Node splitting	No	No	Yes
Typical environment	Main memory	Main memory	Secondary storage
Database indexing	Not ideal	Not normally preferred	Very suitable
Multiple keys per node	No	No	Yes

5.16 The Development of Search Trees

It is useful to understand why these structures are introduced in this order.

Ordinary Binary Search Tree

A BST introduces ordering.

This allows us to avoid examining every element.

However, a BST can become skewed.

Problem:

Worst-case operations can become O(n).

AVL Tree

AVL trees solve the height problem by maintaining balance.

Advantage:

Worst-case search, insertion, and deletion are O(log n).

However, AVL trees are still binary trees, meaning each node has at most two children.

M-Way Search Tree

M-way trees allow nodes to contain multiple keys and many children.

This reduces the height of the tree.

B-Tree

B-trees build on the multiway concept and enforce balancing and occupancy rules.

They are particularly suitable for secondary storage and database indexing.

---

6. IMPORTANT COMPLEXITY SUMMARY

Data Structure	Search	Insert	Delete
Unordered Binary Tree	O(n)	Depends on requirement	O(n)
BST – Average	O(log n)	O(log n)	O(log n)
BST – Worst Case	O(n)	O(n)	O(n)

AVL Tree	O(log n)	O(log n)	O(log n)
B-Tree	O(log n)	O(log n)	O(log n)

For BST and AVL operations, the key relationship is:

$T(n) = O(h)$

where h is the height of the tree.

If:

$h = O(\log n)$

then:

$T(n) = O(\log n)$

If:

$h = O(n)$

then:

$T(n) = O(n)$

---

7. IMPORTANT CONCEPTUAL DISTINCTIONS

Tree vs Binary Tree

A general tree can have an arbitrary number of children.

A binary tree has at most two children.

Binary Tree vs BST

A binary tree does not necessarily have any ordering between its nodes.

A BST maintains an ordering:

Left subtree < Node < Right subtree

BST vs AVL Tree

An AVL tree is a BST that automatically maintains balance.

An ordinary BST does not guarantee logarithmic height.

Binary Tree vs B-Tree

A binary tree has at most two children per node.

A B-tree can have many children and multiple keys per node.

Depth vs Height

Depth:

Distance from root to node.

Height:

Longest distance from node to a leaf.

Preorder vs Inorder vs Postorder

Preorder:

Root -> Left -> Right

Inorder:

Left -> Root -> Right

Postorder:

Left -> Right -> Root

---

8. APPLICATIONS OF TREES

Trees appear in many areas of computer science.

File Systems

A directory structure is naturally hierarchical.

For example:

```
Computer
|
+-- Documents
|   |
|   +-- Assignments
|   +-- Reports
|
+-- Pictures
|   |
|   +-- Family
|   +-- University
|
+-- Music
```

Directories can be viewed as nodes in a tree.

HTML and XML

Web pages and XML documents have hierarchical structures.

For example:

HTML

```
|
+-- HEAD
|
+-- BODY
   |
   +-- H1
   +-- P
   +-- DIV
```

Expression Trees

An arithmetic expression can be represented as a tree.

For:

(a + b) * c

we can construct:

```
     *
    / \
   +   c
  / \
 a   b
```

Different traversals can produce different representations of the expression.

Database Indexing

B-trees and related structures are heavily used in database indexing because they allow efficient searching of very large data sets.

Searching

BSTs and AVL trees provide efficient in-memory searching.

AVL trees are particularly useful when predictable worst-case performance is important.

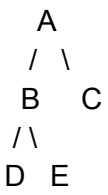
---

9. RECURSION AND TREES

One of the most important programming concepts associated with trees is recursion.

A tree naturally consists of smaller trees.

For example:



The tree can be viewed as:

Tree(A)

containing:

Tree(B)

and:

Tree(C)

Tree(B) itself contains:

Tree(D)

and:

Tree(E)

This recursive structure is why tree algorithms are often naturally implemented recursively.

For example:

```
void inorder(Node* node) {

    if (node == nullptr)
        return;

    inorder(node->left);

    cout << node->data << " ";

    inorder(node->right);
}
```

The function does not need to know the complete structure of the tree.

It simply applies the same rule to each subtree.

This is an important example of how **recursive problem decomposition** works in data structures.

---

10. CORE C++ IMPLEMENTATION SKILLS

After completing this section, students should be able to implement the following in C++.

Basic Binary Tree

```
struct Node {
    int data;
    Node* left;
    Node* right;
};
```

Students should be able to:

- Create nodes.
- Connect nodes.
- Determine whether a node is a leaf.
- Calculate height.
- Calculate depth.
- Traverse the tree.

Traversals

Students should be able to implement:

preorder()
inorder()
postorder()
levelOrder()

and explain the difference between them.

Binary Search Tree

Students should be able to implement:

insert()
search()
findMin()
findMax()
remove()

They should also be able to explain why BST operations depend on tree height.

AVL Tree

Students should understand and implement:

height()
balanceFactor()

rotateLeft()
rotateRight()
insert()
remove()

They should also be able to identify:

LL
RR
LR
RL

imbalances.

B-Trees

Students should understand:

multiple keys per node
multiple children
searching
node splitting
key promotion
redistribution
node merging
balanced height
secondary storage
database indexing

A complete B-tree implementation is significantly more involved than a BST or AVL implementation. The emphasis at this stage should therefore be on understanding the structure, operations, and reasons for its design, unless the course assessment specifically requires a full implementation.

---

11. SUMMARY

A tree is a hierarchical, non-linear data structure consisting of nodes connected by edges.

Important terminology includes:

- Root
- Parent
- Child
- Sibling
- Leaf
- Internal node
- Ancestor
- Descendant
- Subtree
- Depth
- Height
- Degree

Tree traversal allows us to systematically visit every node.

The major traversal methods are:

Preorder:

Root -> Left -> Right

Inorder:

Left -> Root -> Right

Postorder:

Left -> Right -> Root

Level-order:

Visit nodes level by level using a queue.

A binary tree restricts each node to at most two children.

A Binary Search Tree adds an ordering property:

Left subtree < Node < Right subtree

This allows efficient searching when the tree remains reasonably balanced.

However, an ordinary BST can become skewed, resulting in O(n) worst-case operations.

AVL trees solve this problem by maintaining balance using balance factors and rotations.

The four AVL imbalance cases are:

LL -> Right rotation

RR -> Left rotation

LR -> Left rotation followed by right rotation

RL -> Right rotation followed by left rotation

M-way search trees allow nodes to contain multiple keys and multiple children.

B-trees are balanced multiway search trees designed particularly for large data sets and secondary storage.

They are important in:

- Database systems
- Database indexes
- File systems
- Storage systems

The overall progression is therefore:

```
TREE
 |
 +-- Binary Tree
 |     |
 |     +-- Binary Search Tree
 |           |
 |           +-- AVL Tree
 |
 +-- M-Way Search Tree
       |
       +-- B-Tree
```

The key idea students should take away from this topic is:

Trees organize data hierarchically, while search trees exploit that hierarchy to reduce the amount of data that must be examined.

The progression from BST to AVL to B-tree is not arbitrary. Each structure addresses a limitation of the previous one:

